보안 분석 및 공격 방어 메커니즘

개요

파일코인의 저장 증명 시스템은 다양한 공격 벡터에 대응하기 위해 설계된 강력한 보안 메커니즘을 제공합니다. 이 문서에서는 PoRep과 PoSt가 방어하는 주요 공격 유형, 시스템의 암호학적 가정, 그리고 실제 보안 감사 결과를 통해 확인된 보안 강화 사항들을 상세히 분석합니다.

각 Proof 타입이 방어하는 공격 유형

PoRep (Proof-of-Replication) 방어 메커니즘

1. 시간-공간 트레이드오프 공격 (Time-Space Tradeoff Attacks)

공격 개요:

- 공격자가 데이터를 실제로 저장하지 않고 필요할 때 빠르게 재계산하려는 시도
- 저장 공간을 절약하고 계산 능력으로 대체하려는 공격

방어 메커니즘:

```
// 순차적 의존성을 통한 병렬화 방지
parents[predecessor_index] = node - 1; // 즉시 전임자 강제


// Stacked DRG의 다층 구조
pub const LAYERS: usize = 11; // 11개 레이어로 깊이 증가
```

방어 원리:

- **순차적 계산 강제**: 각 노드는 반드시 이전 노드의 계산 완료 후에만 계산 가능
- **깊이 견고성**: 여러 레이어를 통한 깊은 의존성 체인으로 병렬 처리 방지
- **확장 부모**: 이전 레이어와의 연결을 통한 추가적인 계산 복잡도

2. 데이터 압축 공격 (Data Compression Attacks)

공격 개요:

- 원본 데이터를 압축하여 저장 공간을 줄이고 필요시 압축 해제하려는 시도
- 특히 반복적이거나 패턴이 있는 데이터에 대한 공격

방어 메커니즘:

```
// 레이블 생성을 통한 데이터 확장
pub fn create_label<H: Hasher>(
graph: &StackedBucketGraph<H>,
replica_id: &[u8],
layer_labels: &mut [u8],
layer: usize,
node: usize,
) -> Result<()> {
// 부모 노드들의 데이터를 해시하여 새로운 레이블 생성
let label = hash_parents_and_replica_id(parents, replica_id, node);
write_node_label(layer_labels, node, &label);
Ok(())
}
```

방어 원리:

- **의사 랜덤 확장**: 원본 데이터를 해시 기반으로 확장하여 압축 불가능한 형태로 변환
- **레이어별 인코딩**: 각 레이어마다 새로운 레이블 생성으로 압축률 최소화
- **부모 의존성**: 여러 부모 노드의 조합으로 예측 불가능한 데이터 생성

3. 외부 소싱 공격 (Outsourcing Attacks)

공격 개요:

- 저장 제공자가 데이터를 다른 곳에 저장하고 필요시 가져오려는 시도
- 네트워크 지연을 이용한 데이터 접근 시간 단축 시도

방어 메커니즘:

```
// 고유한 복제본 생성
pub fn generate_replica_id(
prover_id: &[u8; 32],
sector_id: u64,
ticket: &[u8; 32],
comm_d: &[u8; 32],
porep_seed: &[u8; 32],
) -> [u8; 32] {
let mut hasher = Sha256::new();
hasher.update(prover_id);
hasher.update(&sector_id.to_le_bytes());
hasher.update(ticket);
hasher.update(comm_d);
```

```
hasher.update(porep_seed);

let mut replica_id = [0u8; 32];

replica_id.copy_from_slice(&hasher.finalize()[..]);

replica_id

}
```

방어 원리:

- **고유한 복제본**: 각 저장 제공자마다 고유한 replica_id로 서로 다른 인코딩 생성
- **증명자별 차별화**: prover_id를 포함하여 동일한 데이터라도 다른 결과 생성
- **시간 제약**: 실시간 증명 생성 요구로 외부 소싱 시간적 불가능

PoSt (Proof-of-Space-time) 방어 메커니즘

1. 생성 공격 (Generation Attacks)

공격 개요:

- 저장 제공자가 데이터를 삭제하고 PoSt 요청 시에만 임시로 재생성하려는 시도
- 저장 공간 절약을 위한 주기적 데이터 재생성

방어 메커니즘:

```
// 무작위 challenge 생성

pub fn generate_sector_challenge<T: Domain>(

randomness: T, // 예측 불가능한 체인 무작위성

n: usize,

sector_set_len: u64,

prover_id: T,
```

```
) -> Result<u64> {
let mut hasher = Sha256::new();
hasher.update(AsRef::<[u8]>::as_ref(&prover_id));
hasher.update(AsRef::<[u8]>::as_ref(&randomness));
hasher.update(&n.to_le_bytes()[..]);

let hash = hasher.finalize();
let sector_challenge = LittleEndian::read_u64(&hash[..8]);
Ok(sector_challenge % sector_set_len)
}
```

방어 원리:

- **예측 불가능한 Challenge**: 체인에서 제공하는 무작위성으로 미리 예측 불가능
- **짧은 증명 시간**: Window PoSt 30분, Winning PoSt 30초 내 증명 생성 요구
- **무작위 샘플링**: 전체 데이터에서 무작위 위치 검증으로 부분 저장 방지

2. 사전 계산 공격 (Precomputation Attacks)

공격 개요:

- 미래의 challenge를 예측하여 미리 증명을 계산해두려는 시도
- 실제 데이터 없이 사전 계산된 증명으로 속이려는 공격

방어 메커니즘:

```
// 체인 무작위성과 현재 에포크 연동
pub fn generate_leaf_challenge<T: Domain>(
```

```
pub_params: &PublicParams,

randomness: T, // 현재 에포크의 무작위성

sector_id: u64,

leaf_challenge_index: u64,

) -> u64 {

let mut hasher = Sha256::new();

hasher.update(AsRef::<[u8]>::as_ref(&randomness));

hasher.update(&sector_id.to_le_bytes()[..]);

hasher.update(&leaf_challenge_index.to_le_bytes()[..]);

let hash = hasher.finalize();

let leaf_challenge = LittleEndian::read_u64(&hash[..8]);

leaf_challenge % (pub_params.sector_size / NODE_SIZE as u64)

}
```

방어 원리:

- **실시간 무작위성**: 각 에포크마다 새로운 무작위성으로 사전 계산 불가능
- **증명자별 차별화**: prover_id 포함으로 증명자마다 다른 challenge 생성
- **머클 증명 요구**: 실제 데이터에 대한 머클 증명으로 사전 계산 방지

3. 복제 공격 (Replication Attacks)

공격 개요:

- 여러 저장 제공자가 동일한 데이터를 공유하여 저장 공간 절약 시도
- 하나의 물리적 저장소를 여러 저장 제공자가 공유하는 공격

방어 메커니즘:

```
// 증명자별 고유한 인코딩
impl<'a, Tree: 'a + MerkleTreeTrait> ProofScheme<'a> for FallbackPoSt<'a, Tree>
fn verify(
pub_params: &Self::PublicParams,
pub_inputs: &Self::PublicInputs,
partition_proof: &Self::Proof,
) -> Result<bool> {
// CommR 검증: H(CommC || CommRLast) == CommR
let comm_r_last = inclusion_proofs[0].root();
if hash2(&comm_c, &comm_r_last) != pub_sector.comm_r {
return Ok(false);
}
// 각 증명자의 고유한 CommR로 복제 방지
}
}
```

방어 원리:

- **고유한 CommR**: 각 저장 제공자마다 다른 CommR로 복제 탐지 가능
- **Replica ID 차별화**: 동일한 원본 데이터라도 저장 제공자별로 다른 인코딩
- **증명 검증**: 실제 인코딩된 데이터에 대한 증명으로 공유 저장 방지

암호학적 가정과 보안 모델

1. 해시 함수 보안 가정

Random Oracle Model (무작위 오라클 모델)

가정 내용:

- SHA256, Blake2s, Poseidon 해시 함수가 이상적인 무작위 함수로 동작
- 해시 함수의 출력이 입력과 독립적으로 균등하게 분포

보안 속성:

```
// 해시 함수의 일방향성 활용
fn create_key(
&self,
id: &H::Domain,
node: usize,
parents: &[u32],
parents_data: &[u8],
exp_parents_data: Option<&[u8]>,
) -> Result<Self::Key> {
let mut hasher = Sha256::new();
hasher.update(AsRef::<[u8]>::as_ref(id));
// 부모 데이터 해시로 역산 불가능한 키 생성
for parent in parents.iter() {
let offset = data_at_node_offset(*parent as usize);
hasher.update(&parents_data[offset..offset + NODE_SIZE]);
}
```

```
let  hash = hasher.finalize();

Ok(bytes_into_fr_repr_safe(hash.as_ref()).into())

}
```

의존하는 속성:

- **일방향성**: 해시 출력으로부터 입력 역산 불가능
- **충돌 저항성**: 동일한 해시값을 가지는 서로 다른 입력 찾기 어려움
- **의사 무작위성**: 출력이 무작위 분포를 따름

해시 함수별 특화 가정

SHA256:

- NIST 표준으로 광범위한 보안 분석 완료
- 2^128 충돌 저항성, 2^256 역상 저항성

Blake2s:

- ChaCha 기반 설계로 높은 보안성
- 소프트웨어 구현에서 SHA256보다 빠른 성능

Poseidon:

- zk-SNARK 회로에 최적화된 설계
- BLS12-381 필드에서 직접 연산으로 효율성 극대화

2. 그래프 이론적 가정

Depth Robust Graph (DRG) 속성

확장 속성 (Expansion Property):

```
// 버킷 샘플링을 통한 확장 보장
```

```
let n_buckets = (metagraph_node as f64).log2().ceil() as u64;

let bucket_index = (rng.gen::<u64>() % n_buckets) + 1;

let largest_distance_in_bucket = min(metagraph_node, 1 << bucket_index);
```

가정 내용:

- 그래프의 임의의 부분집합이 충분히 많은 이웃을 가짐
- 작은 집합이라도 큰 경계(boundary)를 가져야 함

보안 의미:

- 일부 노드 삭제 시에도 나머지 노드들의 연결성 유지
- 공격자가 일부 데이터를 삭제해도 전체 재계산 필요

Feistel 네트워크 가정

의사 무작위 순열 (Pseudorandom Permutation):

```
pub const FEISTEL_ROUNDS: usize = 3;

// 3라운드 Feistel이 충분한 의사 무작위성 제공
```

가정 내용:

- 3라운드 이상의 Feistel 구조가 의사 무작위 순열 생성
- 라운드 함수가 의사 무작위 함수로 동작

보안 의미:

- 확장 부모 선택의 예측 불가능성
- 공격자가 부모-자식 관계를 조작하기 어려움

3. 계산 복잡도 가정

이산 로그 문제 (Discrete Logarithm Problem)

가정 내용:

- BLS12-381 타원곡선에서 이산 로그 계산이 어려움
- zk-SNARK 증명의 건전성 보장

보안 의미:

```
// BLS12-381 스칼라 필드에서의 연산
pub type Fr = blstrs::Scalar;


// 이산 로그 어려움을 기반으로 한 영지식 증명
pub fn verify_proof(
vk: &VerifyingKey,
proof: &Proof,
public_inputs: &[Fr],
) -> Result<bool> {
// Groth16 검증 알고리즘
// e(A, B) = e(α, β) * e(L, γ) * e(C, δ)
}
```

Sequential Work Assumption (순차 작업 가정)

가정 내용:

- 특정 계산은 본질적으로 순차적이며 병렬화 불가능
- 시간을 단축하기 위해서는 더 많은 저장 공간 필요

구현 예시:

```
// 즉시 전임자 의존성으로 순차성 강제
parents[predecessor_index] = node - 1;

// 레이어별 순차 계산
for layer in 1..=LAYERS {
for node in 0..graph.size() {
let label = create_label(graph, replica_id, layer, node)?;
layer_labels[node] = label;
}
}
```

시간-공간 트레이드오프 공격 방어 메커니즘

1. 순차적 계산 강제

즉시 전임자 의존성 (Immediate Predecessor Dependency)

메커니즘:

```
impl<H: Hasher> Graph<H> for BucketGraph<H> {
fn parents(&self, node: usize, parents: &mut [u32]) -> Result<()> {
match node {
0 | 1 => {
// 첫 번째와 두 번째 노드는 자기 참조
```

```
for parent in parents.iter_mut().take(self.degree()) {
*parent = 0;
}
}
_ => {
// 다른 부모들 계산...
// 반드시 바로 이전 노드를 부모로 설정
parents[predecessor_index] = node as u32 - 1;
}
}
Ok(())
}
}
```

방어 효과:

- 각 노드는 이전 노드의 계산 완료를 기다려야 함
- 병렬 처리를 통한 시간 단축 불가능
- 전체 계산 시간이 노드 수에 비례하여 증가

다층 레이어 구조

구현:

```
pub const LAYERS: usize = 11;
```

```
// 각 레이어는 이전 레이어에 의존
for layer in 1..=LAYERS {
if layer == 1 {
// 첫 번째 레이어: 기본 부모만 사용
create_label(graph, cache, replica_id, layer_labels, layer, node)?;
} else {
// 이후 레이어: 기본 + 확장 부모 사용
create_label_exp(graph, cache, replica_id, exp_parents_data,
layer_labels, layer, node)?;
}
}
```

방어 효과:

- 11개 레이어의 순차적 계산으로 깊이 증가
- 중간 레이어 생략 시 최종 결과 계산 불가능
- 저장 공간 절약을 위한 중간 데이터 삭제 방지

2. 메모리 하드 함수 (Memory-Hard Function)

확장 부모를 통한 메모리 접근 패턴

구현:

```
fn correspondent(&self, node: usize, i: usize) -> u32 {
let a = (node * self.expansion_degree) as feistel::Index + i as feistel::Index
// Feistel 순열을 통한 의사 무작위 접근
```

```
let transformed = feistel::permute(
self.size() as feistel::Index * self.expansion_degree as feistel::Index,
a,
&self.feistel_keys,
self.feistel_precomputed,
);
transformed as u32 / self.expansion_degree as u32
}
```

방어 효과:

- 예측 불가능한 메모리 접근 패턴
- 캐시 효율성 저하로 계산 속도 제한
- 전체 데이터에 대한 무작위 접근 강제

부모 캐시 시스템의 제한

구현:

```
pub struct ParentCache {
cache_size: u32,
cache_entries: u32,
cache: Vec<[u32; DEGREE]>,
}

impl ParentCache {
pub fn new(cache_size: u32, cache_entries: u32, graph: &G) -> Result<Self> {
```

```
// 캐시 크기 제한으로 전체 부모 정보 저장 방지
let  cache_size = cache_entries.min(cache_size);
Ok(ParentCache {
cache_size,
cache_entries,
cache: vec![[0u32; DEGREE]; cache_size  as usize],
})
}
}
```

방어 효과:

- 제한된 캐시 크기로 모든 부모 정보 저장 불가능
- 캐시 미스 시 실시간 계산 필요
- 메모리 사용량과 계산 시간의 트레이드오프 강제

3. 시간 제약을 통한 실시간 접근 강제

PoSt 시간 제한

Winning PoSt:

- 30초 내 증명 생성 필요
- 블록 생성 기회 상실 위험

Window PoSt:

- 30분 증명 윈도우 제공
- 실패 시 저장 파워 감소

구현:

```
// 실시간 challenge 생성으로 사전 준비 방지
pub fn generate_leaf_challenge<T: Domain>(
pub_params: &PublicParams,
randomness: T, // 현재 에포크의 무작위성
sector_id: u64,
leaf_challenge_index: u64,
) -> u64 {
// 현재 시점의 무작위성으로 challenge 생성
// 미리 예측하여 준비하는 것 불가능
}
```

방어 효과:

- 실시간 데이터 접근 강제
- 사전 계산된 결과 사용 불가능
- 외부 소싱 시간적 불가능

보안 감사 결과와 해결된 이슈들

1. Protocol Labs 보안 감사 (2020-2021)

주요 발견 사항

이슈 1: Feistel 라운드 수 부족

- **문제**: 초기 2라운드 Feistel이 충분한 보안성 제공하지 못함

- **해결**: 3라운드로 증가하여 의사 무작위성 강화

```
// 수정 전
pub const FEISTEL_ROUNDS: usize = 2;

// 수정 후
pub const FEISTEL_ROUNDS: usize = 3;
// 3 rounds is an acceptable value for a pseudo-random permutation,
// see https://github.com/filecoin-project/rust-proofs/issues/425
```

이슈 2: API 버전별 Challenge 인덱스 불일치

- **문제**: 서로 다른 API 버전에서 challenge 계산 방식 차이
- **해결**: 버전별 명확한 구분과 하위 호환성 보장

```
pub fn get_challenge_index(
api_version: ApiVersion,
sector_index: usize,
challenge_count_per_sector: usize,
challenge_index: usize,
) -> u64 {
(match api_version {
ApiVersion::V1_0_0 | ApiVersion::V1_1_0 => {
sector_index * challenge_count_per_sector + challenge_index
}
```

```
ApiVersion::V1_2_0 => challenge_index,

} as u64)

}
```

보안 강화 조치

1. 부모 노드 순서 표준화

```
// V1.0.0: 즉시 전임자가 마지막 위치

// V1.1.0+: 즉시 전임자가 첫 번째 위치

let predecessor_index = match self.api_version {

ApiVersion::V1_0_0 => m_prime,

ApiVersion::V1_1_0 | ApiVersion::V1_2_0 => 0,

};
```

2. 메타그래프 노드 수 제한

```
// 부동소수점 정밀도 문제 방지

let n_metagraph_nodes = nodes as u64 * m_prime as u64;

ensure!(

n_metagraph_nodes <= 1u64 << 54,

"The number of metagraph nodes must be precisely castable to `f64`"

);
```

2. Sigma Prime 보안 감사 (2021)

주요 발견 사항

이슈 1: 결함 섹터 처리 개선

- **문제**: 결함 섹터 탐지 및 보고 메커니즘 부족
- **해결**: 포괄적인 결함 섹터 처리 시스템 구현

```
// 결함 섹터 탐지 및 수집
let mut faulty_sectors = BTreeSet::new();

match proof {
Ok(proof) => {
if !proof.validate(challenged_leaf as usize) ||
proof.root() != priv_sector.comm_r_last {
error!("Found faulty sector: {:?}", sector_id);
faults.insert(sector_id);
}
}
Err(err) => {
error!("Found faulty sector: {:?}: {:?}", sector_id, err);
faults.insert(sector_id);
}
}

// 결함 섹터가 있으면 오류 반환
```

```
if !faulty_sectors.is_empty() {
return Err(Error::FaultySectors(faulty_sectors.into_iter().collect()));
}
```

이슈 2: 병렬 처리 시 경쟁 조건

- **문제**: 다중 스레드 환경에서 공유 자원 접근 시 경쟁 조건
- **해결**: 스레드 안전한 데이터 구조 사용

```
// 스레드 안전한 결함 섹터 수집
.fold(
|| (Vec::new(), BTreeSet::new()),
|(mut sector_proofs, mut sector_faults), (sector_proof, mut faults)| {
sector_faults.append(&mut faults);
sector_proofs.push(sector_proof);
(sector_proofs, sector_faults)
},
)
.reduce(
|| (Vec::new(), BTreeSet::new()),
|(mut sector_proofs, mut sector_faults), (proofs, mut faults)| {
sector_proofs.extend(proofs);
sector_faults.append(&mut faults);
(sector_proofs, sector_faults)
},
```

```
);
```

3. 지속적인 보안 개선 사항

성능 최적화와 보안성 균형

1. 캐시 크기 제한

```
// 보안성을 위한 캐시 크기 제한
let default_cache_size = SETTINGS.sdr_parents_cache_size;
let cache_size = cache_entries.min(default_cache_size);
```

2. 메모리 접근 패턴 최적화

```
// 프리페치를 통한 성능 향상 (보안성 유지)
fn prefetch(parents: &[u32], data: &[u8]) {
for parent in parents {
let start = *parent as usize * NODE_SIZE;
let end = start + NODE_SIZE;
prefetch!(data[start..end].as_ptr() as *const i8);
}
}
```

암호학적 매개변수 검증

1. 섹터 크기 검증

```
impl ParameterSetMetadata for PublicParams {
fn  sector_size(&self) -> u64 {
(self.nodes * NODE_SIZE) as u64
}
fn  identifier(&self) -> String {
format!(
"drgraph::BucketGraph{{size: {}; degree: {}; hasher: {}}}",
self.nodes,
self.degree(),
H::name(),
)
}
}
```

2. 그래프 매개변수 검증

```
fn  new(
nodes: usize,
base_degree: usize,
expansion_degree: usize,
porep_id: PoRepID,
api_version: ApiVersion,
) -> Result<Self> {
```

```
ensure!(expansion_degree == 0, "Expansion degree must be zero.");

// 메타그래프 노드 수 검증

let  m_prime = base_degree - 1;

let  n_metagraph_nodes = nodes  as u64 * m_prime  as u64;

ensure!(

n_metagraph_nodes <= 1u64 << 54,

"The number of metagraph nodes must be precisely castable to `f64`"

);

Ok(BucketGraph { /* ... */ })

}
```

현재 보안 상태 및 향후 과제

1. 현재 보안 수준

강점:

- 다중 레이어 방어 메커니즘
- 광범위한 보안 감사 완료
- 실전 검증된 암호학적 구성 요소

지속적 모니터링 영역:

- 새로운 공격 벡터 탐지
- 성능 최적화와 보안성 균형
- 하드웨어 발전에 따른 보안 매개변수 조정

2. 향후 보안 강화 방향

양자 컴퓨팅 대응:

- 양자 저항성 해시 함수 연구
- 포스트 양자 암호학 적용 검토

하드웨어 최적화 공격 대응:

- ASIC 저항성 강화 방안
- 메모리 접근 패턴 다양화

확장성과 보안성 균형:

- 더 큰 섹터 크기 지원
- 효율적인 증명 생성 및 검증

결론

파일코인의 저장 증명 시스템은 다양한 공격 벡터에 대한 포괄적인 방어 메커니즘을 제공합니다. PoRep의 시간-공간 트레이드오프 공격 방어, PoSt의 실시간 저장 증명, 그리고 강력한 암호학적 가정을 기반으로 한 보안 모델을 통해 네트워크의 신뢰성을 보장합니다.

지속적인 보안 감사와 개선을 통해 발견된 이슈들이 체계적으로 해결되었으며, 현재는 실전에서 검증된 안전한 저장 증명 시스템으로 운영되고 있습니다. 향후에도 새로운 위협에 대응하고 기술 발전에 맞춰 보안성을 지속적으로 강화해 나갈 예정입니다.